

DML/DDL, Cascade, Bits Conversion, Query Performance

Instructor: Aditya Parameswaran

Covers Lecture(s): 05–06

I DML/DDL

Aditya is trying to order boba for everyone! However, he needs some help with keeping track of boba orders. The student records for enrolled Data 101 students are stored in this table:

```
students (student_id, name, age, major)
```

1. Aditya wants to create an `orders` table that includes `student_id`, `student_name`, `order_id`, `order_info`, and `price`. **All orders should be from enrolled students in `students`.**

Write a SQL command to create this table, where `student_id`, `order_id`, and `price` are integers, and `student_name` and `order_info` are variable-length strings (30 and 50 characters max, respectively). Every order must belong to a student who exists in `students`; enforce this with a constraint.

2. A particular student, Abra Cadabra, contacts course staff to change their boba order. Write a SQL query to update this student's order to 'Passion Fruit Green Tea, light ice, and large'.

3. **[Optional]:** Suppose that, instead of the schema in Q1, `price` is stored as a `VARCHAR`. The `order_info` column contains boba flavor, ice level, and size. For example, the row (000, 'Aditya Parameswaran', 101, 'Brown sugar boba, light ice, large', '\$6') has boba flavor *brown sugar boba*. We want to group the order by boba flavors so it's easier to place the order. In addition, we want to calculate the total price for each flavor.

Write a SQL query that finds the total price and number of orders for each boba flavor.

II Cascade

4. In order to avoid over-ordering bubble tea, Aditya wants to make sure that if a student drop the class, their orders should automatically be dropped from the **orders** table. Alter the foreign key constraint in the **orders** table to implement this behavior.

5. A lucky student, Alex, decides to order themselves not one, but two bobas. The SQL commands are executed like so:

```
INSERT INTO orders (student_id, student_name, order_id, order_info, price)
VALUES (123, 'Alex', 101, 'Mango Boba, regular ice, medium', 5);
```

```
INSERT INTO orders (student_id, student_name, order_id, order_info, price)
VALUES (123, 'Alex', 102, 'Matcha Boba, no ice, large', 6);
```

Does the insertion of the second boba order for Alex violate the foreign key constraint of the **orders** table? Why or why not?

III Memory/Bits Conversion

Suppose each record in your database is 128 bytes, and you have 10 million records. Assume each 8kB page with records has 256 bytes of metadata.

6. Without considering metadata, how large is your database, in SI units?

7. Considering metadata, how many pages do you need to store your database?

IV Query Performance

For this section, we recommend you open DataHub so you can try running these queries yourself in PostgreSQL. For your convenience, we also list the questions here.

This question looks at the impacts of aggregation and sorting on query performance.

8. Write a query that returns the actor names and the number of times the corresponding name appears in the **actors** relation.

9. Write a query that returns the actor IDs and the number of times the corresponding ID appears in the **actors** relation.

10. Run **EXPLAIN ANALYZE** on your two queries above. Why do you think the the **name** query use a Sequential Scan, whereas the **id** query use an Index Only scan?

11. Write a command that creates an index **name_actor_index** on the **name** attribute of the **actors** table. Rerun your **EXPLAIN ANALYZE** of your Question 8 query on **name**. Why does the **name** query now use an Index Only Scan? What index is it using?

12. Now consider sorting. Modify your Question 9 query so the results are ordered by ID (sorted by lowest ID first), and run **EXPLAIN ANALYZE** on it. Do you expect this to take longer than the unsorted version? Why or why not?